String Manipulation

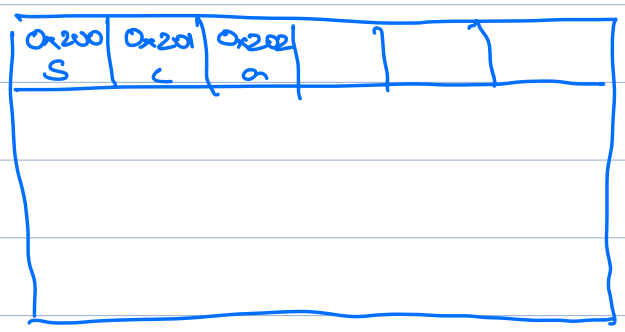
Strings: Sequence of chars.
↳ Array of chars.

'A' $\xrightarrow{\text{ASCII}}$ 65	'a' → 97	'0' → 48
'B' → 66	'b' → 98	'1' → 49
--	--	--
'Z' → 90	'z' → 122	'9' → 57

```
print(ord('A'))  # 65

print(chr(65))  # 'A'
```

s = "Scaler" ['S', 'c', 'a', 'l', 'e', 'r']
↳ 0x200



Question: Given a string consisting of only alphabets print all characters such that:

- every lowercase becomes uppercase
- every uppercase becomes lowercase.

ex: s = "Hello" s = "aDgbHJe"

O/P = "hELLO" O/P = "AdGBhjE"

A → 65 ⇒ a → 97 ⇒ Diff = 32

B → 66 ⇒ b → 98 ⇒ Diff = 32

- - .

Z → 90 ⇒ z → 122 ⇒ Diff = 32

```
def toggleCase(s):
    res = ""
    for char in s:  # N iterations.
        if (65 <= ord(char) <= 97):
        # 'A' <= char <= 'Z'
            new_char = chr(ord(char)+32)
        else:
            new_char = chr(ord(char)-32)
```

```
    res += new_char   # O(N)
return res.
```

$TC = O(N)$ $SC = O(1)$ ✗

$TC = O(N^2)$ $SC = O(N^2)$

<u>String immutability</u> : Strings can't be altered after creation

```
s = "Hello"   # 0x100
s += " World"  # 0x200
print(s)  # "Hello World"
```

0x100 → "Hello"	
0x200 → "Hello World"	

```
s = ""
s += "a"   # "a"      1
s += "b"   # "ab"     ...
s += "c"   # "abc"    N-3
s += "d"   # "abcd"   N-2
s += "e"   # "abcde"  N-1
s += "f"   # "abcdef" N
```

$$\frac{N*(N+1)}{2} = O(N^2)$$

```
def toggleCase(s):
    res = []        # N space
    for char in s:  # N iterations.
        if (65 <= ord(char) <= 97):
        # 'A' <= char <= 'Z'
            new_char = chr(ord(char)+32)
        else:
            new_char = chr(ord(char)-32)
        res.append(new_char)  # O(1)
    return "".join(res)  # O(N)
```

TC = O(N) SC = O(N)

<u>Substring</u> : Contignous part of a string.

"abc" → "ab", "bc", "abc", "a", "b", "c".

<u>"ac" is not a substring.</u>

"bxcd" → "b", "x", "c", "d", "bx", "xc", "cd",
"bxc", "xcd", "bxcd" → 10.

$\boxed{\text{N-length string} \rightarrow N*(N+1)/2}$

Question: Check whether the given substring of string s is palindrome or not

Palindrome → A string which is read the same from left to right or right to left.

0 1 2 3 4 5 6 7 8 9 10

s = "anamadamspe"

st = 3 e = 7

O/P = True.

Brute Force Approach

- Slice the substring # O(N) time # O(N) space
- Reverse it. # O(N) time # O(N) space.
- Check if it is same as the original string. # O(N) time. # O(1) space

$\boxed{TC = O(N) \qquad SC = O(N)}$

0 1 2 3 4 5 6 7 8 9 10

s = "anamadamspe"

i ↗ ↖ j

```
def isSubstringPalindrome(s, st, e):
    i = st , j = e
    while (i < j):
        if(s[i] != s[j]): return False.
        i += 1
        j -= 1
    return True.
```

TC = O(N) SC = O(1)

Question: Given a string find the length of the largest palindromic substring.

0 1 2 3 4 5 6 7 8 9 10

s = "anamadamspe"

O/P = 5

Brute Force Approach

- Generate all substrings # $O(N^2)$
- Check if each substring is palindromic # $O(N)$
- Store the length of the largest palindromic substring. # $O(N)$

$TC = O(N^3)$ $SC = O(1)$

0 1 2 3 4 5 6 7 8 9 10

s = "anamadamspe" → Odd center.

0 1 2 3 4 5 6 7 8 9 10

s = "anamaddamspe" → Even center

C1 C2

Approach

→ If given the center, initilize two pointers at the center and keep checking if the characters are match and grow them.

→ If the logic breaks, then $[i+1, j-1]$ is the length of the palindrome

→ Assume every character as center and repeat.

→ For each character, there's odd length PS and even length PS.

- For odd length, start pointers at same character
- For even length, start one pointer at cur character and the other pointer at next char.

→ Store the length of largest PS.

```
def longestPalindromicSubstring(s):
    ans = 0
    for c in range(len(s)):        # N iterations
        # odd length center
        odd_len = PSAroundCenter(s,c,c)   # O(N)
        # even length center
        even_len = PSAroundCenter(s, c, c+1)  # O(N)
        ans = max(ans, odd_len, even_len)
    return ans
```

$[a, b] \rightarrow b-a+1$; $[i+1, j-1] \rightarrow j-1-i\cancel{-1}\cancel{+1}$

```
def PSAroundCenter(s, i, j):
    while (i>=0 and j<len(s)):   # O(N)
        if(s[i] != s[j]) : return j-i-1
        i-=1
        j+=1
    retur j-i-1
```

$TC = O(N^2)$ $SC = O(1)$

0 1 2 3 4 5 6 7 8 9 10

s = "anamadamspe"

c

odd_len = [2, 8] = 8 - 2 - 1 = 5

even_len = 0

ans = ~~0~~ ~~1~~ ~~2~~ 5

0 1 2 3 4 5 6 7 8 9 10

s = "anamaddamspe"

c

odd_len = 1

even_len = [2, 9] = 9 - 2 - 1 = 6

ans = ~~0~~ ~~1~~ ~~2~~ 6

<u>Question</u>: Given an input string, reverse the order of the words.

eg: s = "Scaler is the best"

O/P = "best the is Scaler"

s = "hello world"

O/P = "world hello".

```
def reverseWords(s):
    l = s.split()
    l.reverse()
    return " ".join(l)
```

TC = O(N) SC = O(N)